8.2.4 브릿지 연결 구현하기

개요

이 고급 튜토리얼에서는 Substrate 기반 체인과 외부 블록체인(이더리움) 간의 브릿지를 구현합니다. 양방향 자산 전송, 상태 동기화, 보안 검증 메커니즘을 포함한 완전한 크로스체인 브릿지 시스템을 구축합니다.

학습 목표

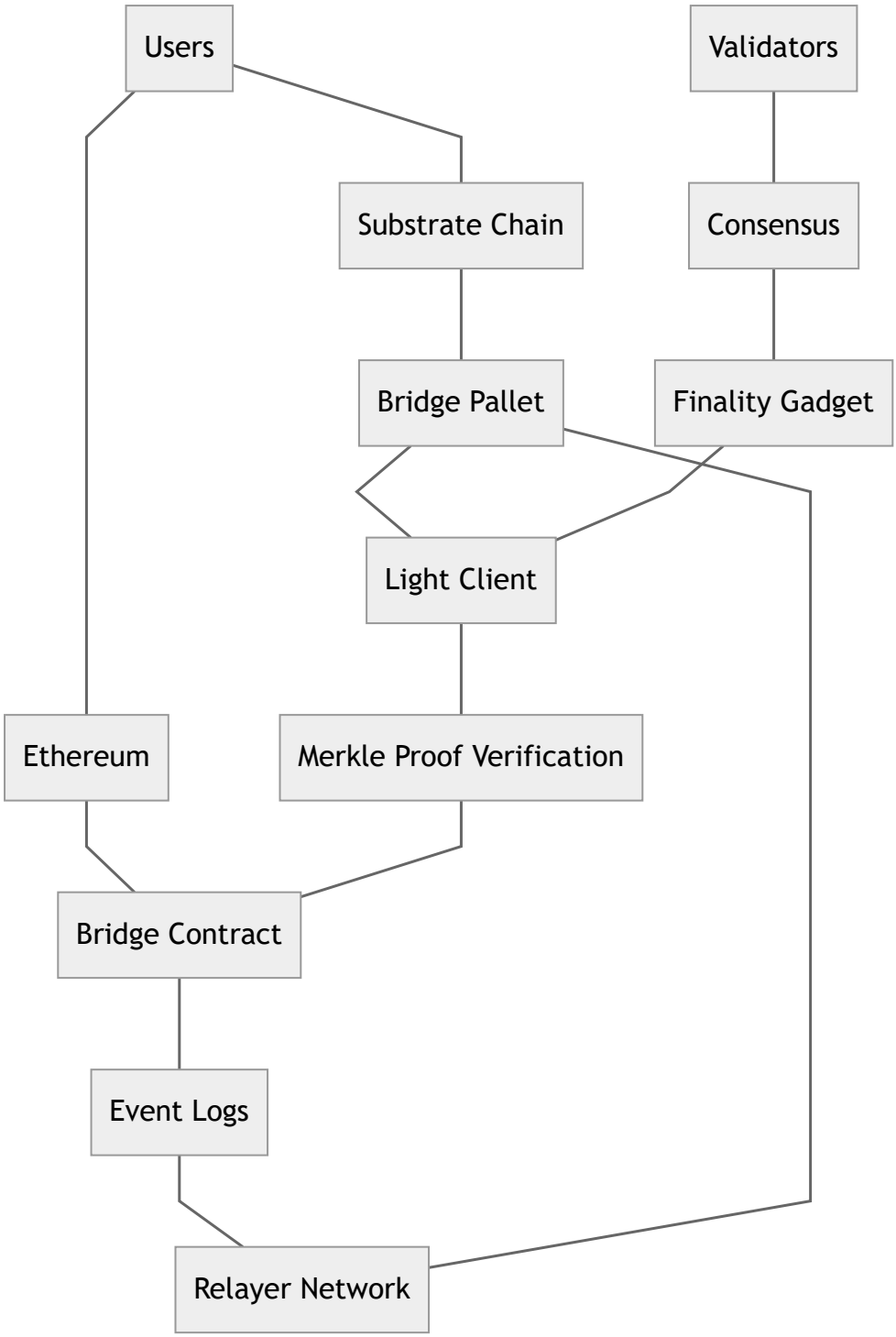
- 브릿지 아키텍처 설계 및 구현
- 라이트 클라이언트 기반 검증 시스템
- 릴레이어 네트워크 구축
- 이더리움 스마트 컨트랙트 연동
- 보안 모델 및 경제적 인센티브 설계

사전 요구사항

- 고급 Substrate 개발 경험
- 이더리움 스마트 컨트랙트 개발 지식
- 암호학적 증명 시스템 이해
- 분산 시스템 설계 경험

1단계: 브릿지 아키텍처 설계

1.1 시스템 구조



1.2 핵심 컴포넌트

```
// pallets/ethereum-bridge/src/lib.rs

#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]

pub struct EthereumHeader {
```

```
pub parent_hash: H256,
pub uncle_hash: H256,
pub author: H160,
pub state_root: H256,
pub transactions_root: H256,
pub receipts_root: H256,
pub log_bloom: Bloom,
pub difficulty: U256,
pub number: U256,
pub gas_limit: U256,
pub gas_used: U256,
pub timestamp: u64,
pub extra_data: Vec<u8>,
pub mix_hash: H256,
pub nonce: H64,
}

#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub struct BridgeTransfer<AccountId, Balance> {
pub from: AccountId,
pub to: Vec<u8>, // Ethereum address
pub amount: Balance,
pub token_address: H160,
```

```
pub nonce: u64,
pub ethereum_tx_hash: Option<H256>,
}

#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub enum TransferStatus {
Pending,
Submitted,
Confirmed,
Failed,
}
```

2단계: 라이트 클라이언트 구현

2.1 이더리움 라이트 클라이언트

```
impl<T: Config> Pallet<T> {
/// 이더리움 헤더 검증 및 저장
pub fn verify_and_store_header(
header: EthereumHeader,
proof: Vec<Vec<u8>>,
) -> DispatchResult {
// 1. 헤더 유효성 검증
```

```
Self::verify_header_validity(&header)?;

// 2. PoW 검증

Self::verify_proof_of_work(&header)?;

// 3. 체인 연속성 확인

Self::verify_chain_continuity(&header)?;

// 4. 헤더 저장

EthereumHeaders::<T>::insert(&header.number, &header);

// 5. 최신 헤더 업데이트

if header.number > Self::latest_ethereum_block() {

LatestEthereumBlock::<T>::put(&header.number);

}

Ok(())

}



/// PoW 검증

fn verify_proof_of_work(header: &EthereumHeader) -> DispatchResult {

let hash = Self::hash_header(header);

let target = Self::calculate_difficulty_target(header.difficulty);

ensure!(hash <= target, Error::<T>::InvalidProofOfWork);

Ok(())

}



/// 머클 증명 검증
```

```
pub fn verify_merkle_proof(
root: H256,
proof: &[H256],
leaf: H256,
index: u64,
) -> bool {
let mut computed_hash = leaf;
let mut current_index = index;
for proof_element in proof {
if current_index % 2 == 0 {
computed_hash = Self::keccak256(&[computed_hash.as_bytes(), proof_element.as_byt
} else {
computed_hash = Self::keccak256(&[proof_element.as_bytes(), computed_hash.as_byt
}
current_index /= 2;
}
computed_hash == root
}
}
```

3단계: 자산 전송 구현

3.1 Substrate → Ethereum 전송

```
#[pallet::call]
impl<T: Config> Pallet<T> {
/// 이더리움으로 자산 전송
#[pallet::call_index(0)]
#[pallet::weight(10_000)]
pub fn transfer_to_ethereum(
origin: OriginFor<T>,
ethereum_address: H160,
amount: T::Balance,
token_address: H160,
) -> DispatchResult {
let sender = ensure_signed(origin)?;
// 잔액 확인
ensure!(
T::Currency::free_balance(&sender) >= amount,
Error::<T>::InsufficientBalance
);
// 전송 ID 생성
let transfer_id = Self::next_transfer_id();
NextTransferId::<T>::put(transfer_id + 1);
// 자산 잠금
```

```
T::Currency::reserve(&sender, amount)?;

// 전송 기록 생성

let  transfer = BridgeTransfer {

from: sender.clone(),

to: ethereum_address.as_bytes().to_vec(),

amount,

token_address,

nonce: transfer_id,

ethereum_tx_hash: None,

};

PendingTransfers::<T>::insert(&transfer_id, &transfer);

TransferStatuses::<T>::insert(&transfer_id, TransferStatus::Pending);

// 릴레이어에게 전송 요청 이벤트 발생

Self::deposit_event(Event::TransferRequested {

transfer_id,

from: sender,

to: ethereum_address,

amount,

token_address,

});

Ok(())

}
```

```
/// 이더리움 전송 확인
#[pallet::call_index(1)]
#[pallet::weight(10_000)]
pub fn confirm_ethereum_transfer(
origin: OriginFor<T>,
transfer_id: u64,
ethereum_tx_hash: H256,
block_number: U256,
tx_index: u64,
proof: Vec<Vec<u8>>,
) -> DispatchResult {
ensure_signed(origin)?;
// 전송 기록 확인
let transfer = Self::pending_transfers(&transfer_id)
.ok_or(Error::<T>::TransferNotFound)?;
// 이더리움 블록 헤더 확인
let header = Self::ethereum_headers(&block_number)
.ok_or(Error::<T>::HeaderNotFound)?;
// 트랜잭션 영수증 검증
Self::verify_transaction_receipt(
&header.receipts_root,
ethereum_tx_hash,
tx_index,
```

```
&proof,
&transfer,
)?;
// 상태 업데이트
TransferStatuses::<T>::insert(&transfer_id, TransferStatus::Confirmed);
PendingTransfers::<T>::remove(&transfer_id);
// 이벤트 발생
Self::deposit_event(Event::TransferConfirmed {
transfer_id,
ethereum_tx_hash,
});
Ok(())
}
}
```

3.2 Ethereum → Substrate 전송

```
impl<T: Config> Pallet<T> {
/// 이더리움에서 오는 전송 처리
#[pallet::call_index(2)]
#[pallet::weight(10_000)]
pub fn process_ethereum_transfer(
origin: OriginFor<T>,
```

```
ethereum_tx_hash: H256,
block_number: U256,
log_index: u64,
proof: Vec<Vec<u8>>,
recipient: T::AccountId,
amount: T::Balance,
token_address: H160,
) -> DispatchResult {
ensure_signed(origin)?;
// 중복 처리 방지
ensure!(
!ProcessedTransfers::<T>::contains_key(&ethereum_tx_hash),
Error::<T>::TransferAlreadyProcessed
);
// 이더리움 블록 헤더 확인
let  header = Self::ethereum_headers(&block_number)
.ok_or(Error::<T>::HeaderNotFound)?;
// 로그 이벤트 검증
Self::verify_transfer_log(
&header.receipts_root,
ethereum_tx_hash,
log_index,
&proof,
```

```
&recipient,
amount,
token_address,
)?;
// 자산 발행 또는 잠금 해제
Self::mint_or_unlock_tokens(&recipient, amount, token_address)?;
// 처리 완료 기록
ProcessedTransfers::<T>::insert(&ethereum_tx_hash, true);
// 이벤트 발생
Self::deposit_event(Event::TransferProcessed {
ethereum_tx_hash,
recipient,
amount,
token_address,
});
Ok(())
}


/// 로그 이벤트 검증
fn  verify_transfer_log(
receipts_root: &H256,
tx_hash: H256,
log_index: u64,
```

```
proof: &[Vec<u8>],
recipient: &T::AccountId,
amount: T::Balance,
token_address: H160,
) -> DispatchResult {
// 영수증 머클 증명 검증
let  receipt_proof = &proof[0];
let  log_proof = &proof[1];
// 1. 트랜잭션 영수증 검증
let  receipt_hash = Self::keccak256(receipt_proof);
ensure!(
Self::verify_merkle_proof(*receipts_root, &[], receipt_hash, log_index),
Error::<T>::InvalidReceiptProof
);
// 2. 로그 데이터 파싱 및 검증
let  log_data = Self::parse_transfer_log(log_proof)?;
ensure!(log_data.recipient == *recipient, Error::<T>::InvalidRecipient);
ensure!(log_data.amount == amount, Error::<T>::InvalidAmount);
ensure!(log_data.token == token_address, Error::<T>::InvalidToken);
Ok(())
}
}
```

4단계: 이더리움 스마트 컨트랙트

4.1 브릿지 컨트랙트

```
// contracts/SubstrateBridge.sol

pragma solidity ^0.8.0;


import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

import "@openzeppelin/contracts/access/Ownable.sol";


contract SubstrateBridge is ReentrancyGuard, Ownable {

struct Transfer {

address from;

bytes32 to; // Substrate account ID

uint256 amount;

address token;

uint64 nonce;

bool processed;

}

mapping(uint64 => Transfer) public transfers;

mapping(bytes32 => bool) public processedSubstrateTransfers;

mapping(address => bool) public supportedTokens;
```

```
uint64 public nextNonce = 1;

event TransferToSubstrate(

uint64 indexed nonce,

address indexed from,

bytes32 indexed to,

uint256 amount,

address token

);

event TransferFromSubstrate(

bytes32 indexed substrateHash,

address indexed to,

uint256 amount,

address token

);

modifier onlyRelayer() {

require(relayers[msg.sender], "Not authorized relayer");

_;

}

/// Substrate로 자산 전송

function transferToSubstrate(

bytes32 substrateAccount,

uint256 amount,

address token
```

```
) external nonReentrant {

require(supportedTokens[token], "Token not supported");

require(amount > 0, "Amount must be positive");

// 토큰 잠금

IERC20(token).transferFrom(msg.sender, address(this), amount);

// 전송 기록

uint64 nonce = nextNonce++;

transfers[nonce] = Transfer({

from: msg.sender,

to: substrateAccount,

amount: amount,

token: token,

nonce: nonce,

processed: false

});

emit TransferToSubstrate(nonce, msg.sender, substrateAccount, amount, token);

}

/// Substrate에서 오는 전송 처리

function processSubstrateTransfer(

bytes32 substrateHash,

address to,

uint256 amount,

address token,
```

```
bytes calldata proof

) external onlyRelayer nonReentrant {

require(!processedSubstrateTransfers[substrateHash], "Already processed");

require(supportedTokens[token], "Token not supported");

// Substrate 증명 검증 (실제 구현에서는 더 복잡)

require(verifySubstrateProof(substrateHash, to, amount, token, proof), "Invalid

// 토큰 전송

IERC20(token).transfer(to, amount);

// 처리 완료 기록

processedSubstrateTransfers[substrateHash] = true;

emit TransferFromSubstrate(substrateHash, to, amount, token);

}

/// Substrate 증명 검증 (간소화된 버전)

function verifySubstrateProof(

bytes32 substrateHash,

address to,

uint256 amount,

address token,

bytes calldata proof

) internal pure returns (bool) {

// 실제 구현에서는 Substrate 블록 헤더와 머클 증명을 검증

// 여기서는 간소화된 버전

bytes32 computedHash = keccak256(abi.encodePacked(to, amount, token));
```

```
return computedHash == substrateHash;

}

/// 지원 토큰 추가

function addSupportedToken(address token) external onlyOwner {

supportedTokens[token] = true;

}

/// 릴레이어 관리

mapping(address => bool) public relayers;

function addRelayer(address relayer) external onlyOwner {

relayers[relayer] = true;

}

function removeRelayer(address relayer) external onlyOwner {

relayers[relayer] = false;

}

}
```

5단계: 릴레이어 시스템

5.1 릴레이어 구현

```
// relayer/src/main.rs

use std::time::Duration;

use tokio::time::sleep;
```

```rust
use web3::{Web3, transports::Http};
use substrate_api_client::{Api, XtStatus};

pub struct BridgeRelayer {
substrate_api: Api<sr25519::Pair>,
ethereum_web3: Web3<Http>,
ethereum_contract: H160,
relayer_account: sr25519::Pair,
}

impl BridgeRelayer {
pub async fn new() -> Result<Self, Box<dyn std::error::Error>> {
// Substrate API 연결
let substrate_api = Api::new("ws://localhost:9944").await?;
// Ethereum Web3 연결
let transport = Http::new("http://localhost:8545")?;
let ethereum_web3 = Web3::new(transport);
// 릴레이어 계정 로드
let relayer_account = sr25519::Pair::from_string("//Alice", None)?;
Ok(Self {
substrate_api,
ethereum_web3,
ethereum_contract: "0x1234...".parse()?,
```

```
relayer_account,
})
}

/// 메인 릴레이 루프
pub  async  fn  run(&self) -> Result<(), Box<dyn  std::error::Error>> {
loop {
// Substrate → Ethereum 전송 처리
self.process_substrate_to_ethereum().await?;
// Ethereum → Substrate 전송 처리
self.process_ethereum_to_substrate().await?;
// 이더리움 헤더 동기화
self.sync_ethereum_headers().await?;
sleep(Duration::from_secs(12)).await; // 블록 시간에 맞춤
}
}

/// Substrate → Ethereum 전송 처리
async  fn  process_substrate_to_ethereum(&self) -> Result<(), Box<dyn  std::erro
// Substrate에서 전송 요청 이벤트 모니터링
let  events = self.substrate_api.get_events().await?;
for  event  in  events {
if  let Some(transfer_event) = self.parse_transfer_request(&event) {
```

```
// 이더리움에서 전송 실행
self.execute_ethereum_transfer(transfer_event).await?;
}
}
Ok(())
}


/// Ethereum → Substrate 전송 처리
async fn process_ethereum_to_substrate(&self) -> Result<(), Box<dyn std::erro
// 이더리움 컨트랙트 이벤트 모니터링
let latest_block = self.ethereum_web3.eth().block_number().await?;
let from_block = latest_block - 100; // 최근 100블록 확인
let filter = FilterBuilder::default()
.address(vec![self.ethereum_contract])
.from_block(BlockNumber::Number(from_block))
.to_block(BlockNumber::Number(latest_block))
.topics(
Some(vec![
"0x1234...".parse()? // TransferToSubstrate 이벤트 시그니처
]),
None,
None,
None,
```

```
)
.build();
let logs = self.ethereum_web3.eth().logs(filter).await?;
for log in logs {
if let Some(transfer_data) = self.parse_ethereum_transfer_log(&log) {
// Substrate에서 전송 처리
self.process_substrate_transfer(transfer_data).await?;
}
}
Ok(())
}


/// 이더리움 헤더 동기화
async fn sync_ethereum_headers(&self) -> Result<(), Box<dyn std::error::Error
let latest_block = self.ethereum_web3.eth().block_number().await?;
let substrate_latest = self.get_substrate_latest_ethereum_block().await?;
// 누락된 헤더들 동기화
for block_num in (substrate_latest + 1)..=latest_block.as_u64() {
let block = self.ethereum_web3.eth()
.block(BlockId::Number(BlockNumber::Number(block_num.into())))
.await?
.ok_or("Block not found")?;
// Substrate에 헤더 제출
```

```
let call = self.substrate_api.compose_call(
"EthereumBridge",
"submit_ethereum_header",
(block.header, vec![] as Vec<Vec<u8>>), // 간소화된 증명
)?;
self.substrate_api.send_extrinsic(call, XtStatus::InBlock).await?;
}
Ok(())
}
}


#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
let relayer = BridgeRelayer::new().await?;
relayer.run().await
}
```

6단계: 보안 및 경제적 인센티브

6.1 슬래싱 메커니즘

```
impl<T: Config> Pallet<T> {
/// 릴레이어 등록
```

```
#[pallet::call_index(3)]

#[pallet::weight(10_000)]

pub fn register_relayer(

origin: OriginFor<T>,

stake: T::Balance,

) -> DispatchResult {

let relayer = ensure_signed(origin)?;

// 최소 스테이크 확인

ensure!(stake >= T::MinRelayerStake::get(), Error::<T>::InsufficientStake);

// 스테이크 잠금

T::Currency::reserve(&relayer, stake)?;

// 릴레이어 등록

Relayers::<T>::insert(&relayer, RelayerInfo {

stake,

reputation: 100, // 초기 평판

last_activity: frame_system::Pallet::<T>::block_number(),

});

Ok(())

}



/// 잘못된 릴레이 보고

#[pallet::call_index(4)]

#[pallet::weight(10_000)]
```

```
pub fn report_malicious_relayer(
origin: OriginFor<T>,
relayer: T::AccountId,
evidence: Vec<u8>,
) -> DispatchResult {
let reporter = ensure_signed(origin)?;
// 증거 검증
Self::verify_malicious_evidence(&relayer, &evidence)?;
// 릴레이어 슬래싱
if let Some(mut relayer_info) = Self::relayers(&relayer) {
let slash_amount = relayer_info.stake / 10u32.into(); // 10% 슬래싱
T::Currency::slash_reserved(&relayer, slash_amount);
relayer_info.stake = relayer_info.stake.saturating_sub(slash_amount);
relayer_info.reputation = relayer_info.reputation.saturating_sub(20);
if relayer_info.stake < T::MinRelayerStake::get() {
// 릴레이어 제거
Relayers::<T>::remove(&relayer);
T::Currency::unreserve(&relayer, relayer_info.stake);
} else {
Relayers::<T>::insert(&relayer, relayer_info);
}
// 신고자 보상
let reward = slash_amount / 2u32.into();
```

```
T::Currency::deposit_creating(&reporter, reward);

}

Ok(())

}

}
```

7단계: 테스트 및 배포

7.1 통합 테스트

```
#[cfg(test)]

mod tests {

use  super::*;

use  crate::mock::*;


#[test]

fn  full_bridge_flow_works() {

new_test_ext().execute_with(|| {

// 1. 릴레이어 등록

assert_ok!(EthereumBridge::register_relayer(

RuntimeOrigin::signed(1),

1000000

));
```

```
// 2. Substrate → Ethereum 전송
assert_ok!(EthereumBridge::transfer_to_ethereum(
RuntimeOrigin::signed(2),
H160::from_low_u64_be(0x1234),
100000,
H160::zero()
));

// 3. 전송 확인
let  transfer_id = 0;
assert_ok!(EthereumBridge::confirm_ethereum_transfer(
RuntimeOrigin::signed(1),
transfer_id,
H256::random(),
U256::from(1000),
0,
vec![]
));

// 4. Ethereum → Substrate 전송 처리
assert_ok!(EthereumBridge::process_ethereum_transfer(
RuntimeOrigin::signed(1),
```

```
H256::random(),

U256::from(1001),

0,

vec![],

3,

50000,

H160::zero()

));

});

}

}
```

요약

이 고급 튜토리얼에서는 다음을 구현했습니다:

1. **완전한 브릿지 시스템**: Substrate-Ethereum 양방향 자산 전송
2. **라이트 클라이언트**: 이더리움 헤더 검증 및 머클 증명 시스템
3. **스마트 컨트랙트**: 이더리움 측 브릿지 로직
4. **릴레이어 네트워크**: 자동화된 크로스체인 메시지 전달
5. **보안 메커니즘**: 슬래싱, 평판 시스템, 경제적 인센티브

참고 자료

- Substrate Bridges

- Snowbridge 구현
- 이더리움 라이트 클라이언트